Module I

operators and expressions in C

By: RISHI KUMAR

C Input and Output (I/O)

- scanf() function to take input from the user and printf() function to display output to the user.
- scanf() and printf() functions are inbuilt library functions in C programming language which are available in C library by default.
- These functions are declared and related macros are defined in “stdio.h” which is a header file in C language.
- We have to include “stdio.h” file as shown in all C program to make use of these scanf() and printf() library functions in C language (it is optional for some compilers).
- Key points to remember in C scanf() and printf():
  - scanf() is used to read the formated inputs and printf() is used to display the formatted output.
  - scanf() and printf() functions are declared and defined in “stdio.h” header file in C library.
  - All syntax in C language including scanf() and printf() functions are case sensitive.

scanf() Function in C Language

- In C programming language, scanf() function is used to read character, string, numeric data from keyboard.
- scanf("format string", argument_list); *i.e.* scanf("%format_specifier", &Variable_Name);
- Consider aside example program where user enters a character. This value is assigned to the variable "ch" and then displayed.
- Then, user enters a string and this value is assigned to the variable "str" and then displayed.

```
#include <stdio.h>
int main()
{
   char ch;
   char str[100];
   printf("Enter any character \n");
   scanf("%c", &ch);
   printf("Entered character is %c \n", ch);
   printf("Enter any string ( upto 100 character ) \n");
   scanf("%s", &str);
   printf("Entered string is %s \n", str);
}
```

- The format specifier %d is used in scanf() statement. So that, the value entered is received as an integer and %s for string.
- Ampersand is used before variable name "ch" in scanf() statement as &ch.

printf() Function in C Language

- In C programming language, printf() function is used to print the (“character, string, float, integer, octal and hexadecimal values”) onto the output screen.
- printf(“Display text”, Variable_Name); *i.e.*

  printf(“Text %format_specifier”, Variable_Name);
- We use printf() function with %d format specifier to display the value of an integer variable.
- Similarly %c is used to display character, %f for float variable, %s for string variable, %lf for double and %x for hexadecimal variable.
- To generate a newline, we use “\n” in C printf() statement.

```
#include <stdio.h>
int main()
{
    char ch = 'A';
    char str[20] = “graphicera.edu";
    float flt = 10.234;
    int no = 150;
    double dbl = 20.123456;
    printf("Character is %c \n", ch);
    printf("String is %s \n" , str);
    printf("Float value is %f \n", flt);
    printf("Integer value is %d\n" , no);
    printf("Double value is %lf \n", dbl);
    printf("Octal value is %o \n", no);
    printf("Hexadecimal value is %x \n", no);
  return 0;
}
```

C Data Types and their Format Specifiers

- The format specifiers are used in C for input and output purposes.
- Using this concept, the compiler can understand that what type of data is in a variable during taking input using the scanf() function and printing using printf() function.
- A number after % specifies the minimum field width. If string is less than the width, it will be filled with spaces.
- Here is a list of format specifiers supported by C:

Data Type	Format Specifier
int	%d
char	%c
float	%f
double	%lf
char	%c
string	%s

Operators in C

An operator is a symbol which helps the user to command the computer to do a certain mathematical or logical manipulations. Operators are used in C language program to operate on data and variables. C has a rich set of operators which can be classified as :

Eight types of operators are in C

- Arithmetic operators
- Assignment operators
- Relational operators
- Logical operators
- Bit wise operators
- Conditional operators (ternary operators)
- Increment/decrement operators
- Special operators

1. Arithmetic Operators

- These are used to perform mathematical calculations like addition, subtraction, multiplication, division and modulus:

Arithmetic Operators/Operation	Example
+ (Addition)	A+B
– (Subtraction)	A-B
* (multiplication)	A*B
/ (Division)	A/B
% (Modulus)	A%B

```
#include<stdio.h>
void main()
{
    int numb1, num2, sum, sub, mul, div, mod;
    scanf (“%d %d”, &num1, &num2); //inputs the operands

    sum = num1+num2;
    printf(“\n The sum is = %d”, sum);

    sub = num1-num2;
    printf(“\n The difference is = %d”, sub);

    mul = num1*num2;
    printf(“\n The product is = %d”, mul);

    div = num1/num2;
    printf(“\n The division is = %d”, div);

    mod = num1%num2;
    printf(“\n The modulus is = %d”, mod);
}
```

2. Assignment Operators

There are 2 categories of assignment operators in C language. They are:

1. Simple assignment operator (Example: =)
2. Compound assignment operators (Example: +=, -=, *=, /=, %=, &=, ^=

Operators	Example/Description
=	sum = 10; 10 is assigned to variable sum
+=	sum += 10; This is same as sum = sum + 10
-=	sum -= 10; This is same as sum = sum – 10
*=	sum *= 10; This is same as sum = sum * 10
/=	sum /= 10; This is same as sum = sum / 10
%=	sum %= 10; This is same as sum = sum % 10
&=	sum&=10; This is same as sum = sum & 10
^=	sum ^= 10; This is same as sum = sum ^ 10

In this program, values from 0 – 9 are summed up and total “45” is displayed as output.
Assignment operators such as “=” and “+=” are used in this program to assign the values and to sum up the values.

```
# include <stdio.h>
 int main()
{
      int Total=0,i;
      for(i=0;i<10;i++)
      {
            Total+=i;        // This is same as Total = Total+i
       }
        printf("Total = %d", Total);
}
```

Output: Total = 45

Statement with simple assignment operator	Statement with shorthand operator
a = a + 1	a += 1
a = a – 1	a -= 1
a = a * (n+1)	a *= (n+1)
a = a / (n+1)	a /= (n+1)
a = a % b	a %= b

```
#include<stdio.h>

main()
{
  int a;
  int A=2, N=100;
  a = A;

  while (a < N)
  {
    printf("%d \n",a);
    a *= a;
  }
}
```

Output

2
4
16
--
--

3. Relational Operators

➢Relational operators are used to find the relation between two variables. *i.e.*

to compare the values of two variables in a C program.

Operators	Example/Description
>	x > y (x is greater than y)
<	x < y (x is less than y)
>=	x >= y (x is greater than or equal to y)
<=	x <= y (x is less than or equal to y)
= =	x == y (x is equal to y)
!=	x != y (x is not equal to y)

- In this program, relational operator (= =) is used to compare 2 values whether they are equal are not.
- If both values are equal, output is displayed as ” values are equal”. Else, output is displayed as “values are not equal”.
- **Note**: double equal sign (==) should be used to compare 2 values. We should not single equal sign (=).

```
#include <stdio.h>
int main()
{
   int m=40,n=20;
   if (m == n)
   {
      printf("m and n are equal");
   }
   else
   {
      printf("m and n are not equal");
   }
}
```

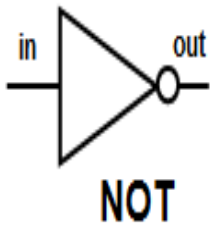
4. Logical Operators

➢ These operators are used to perform logical operations on the given expressions:

- There are 3 logical operators in C language.
- They are, logical AND (&&), logical OR (||) and logical NOT (!).

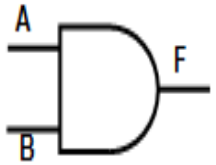
Operators	Example/Description
&& (logical AND)	(x>5)&&(y<5) It returns true when both conditions are true
|| (logical OR)	(x>=10)||(y>=10) It returns true when at-least one of the condition is true
! (logical NOT)	!((x>5)&&(y<5)) It reverses the state of the operand "((x>5) && (y<5))" If "((x>5) && (y<5))" is true, logical NOT operator makes it false

```
#include <stdio.h>
 int main()
{
    int m=40,n=20;
    int o=20,p=30;
    if (m>n && m !=0)
    {
        printf("&& Operator : Both conditions are true\n");
    }
    if (o>p || p!=20)
    {
        printf("|| Operator : Only one condition is true\n");
    }
    if (!(m>n && m !=0))
    {
        printf("! Operator : Both conditions are true\n");
    }
    else
    {
       printf("! Operator : Both conditions are true. " \
       "But, status is inverted as false\n");
    }
 }
```



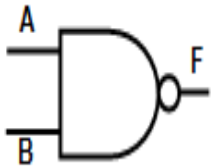
NOT

Input	Output
I	F
0	1
1	0



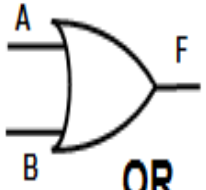
AND

Inputs		Output
A	B	F
0	0	0
1	0	0
0	1	0
1	1	1



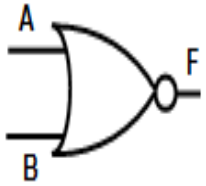
NAND

Inputs		Output
A	B	F
0	0	1
1	0	1
0	1	1
1	1	0



OR

Inputs		Output
A	B	F
0	0	0
1	0	1
0	1	1
1	1	1



NOR

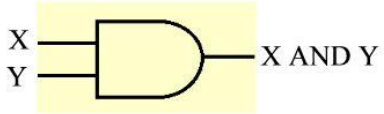
Inputs		Output
A	B	F
0	0	1
1	0	0
0	1	0
1	1	0

Logical AND

Truth table		
X	Y	X AND Y
False	False	False
False	True	False
True	False	False
True	True	True

Truth table		
X	Y	X AND Y
0	0	0
0	1	0
1	0	0
1	1	1

Logic gate



James Tam

5. Bit-wise Operators

- These operators are used to perform bit operations.
- Decimal values are converted into binary values which are sequence of bits and bit wise operators work on these bits.
- Bit wise operators in C language are: & (bitwise AND), | (bitwise OR), ~ (bitwise NOT), ^ (XOR), << (left shift) and >> (right shift).

Operators	Example/Description
&	Bitwise AND
|	Bitwise OR
~	Bitwise NOT
^	XOR
<<	Left Shift
>>	Right Shift


```
#include <stdio.h>
 int main()
{
    int m = 40, n = 80, AND_opr, OR_opr,
XOR_opr, NOT_opr ;
    AND_opr = (m&n);
    OR_opr = (m|n);
    NOT_opr = (~m);
    XOR_opr = (m^n);
    printf("AND_opr value = %d\n", AND_opr
);
    printf("OR_opr value = %d\n", OR_opr );
    printf("NOT_opr value = %d\n", NOT_opr
);
    printf("XOR_opr value = %d\n", XOR_opr
);
    printf("left_shift value = %d\n", m << 1);
    printf("right_shift value = %d\n", m >> 1);
}
```

Output:

```
AND_opr value = 0
OR_opr value = 120
NOT_opr value = -41
XOR_opr value = 120
left_shift value = 80
right_shift value = 20
```

<< (left shift)

Shifts bits to the left. The number to the left of the operator is shifted the number of places specified by the number to the right. Each shift to the left doubles the number, therefore each left shift multiplies the original number by 2. Use the left shift for fast multiplication or to pack a group of numbers together into one larger number. Left shifting only works with integers or numbers which automatically convert to an integer such at byte and char.

```
int m = 1 << 3;   // In binary: 1 to 1000
print(m);  // Prints "8"
int n = 1 << 8;   // In binary: 1 to 100000000
print(n);  // Prints "256"
int o = 2 << 3;   // In binary: 10 to 10000
print(o);  // Prints "16"
int p = 13 << 1;  // In binary: 1101 to 11010
print(p);  // Prints "26“
```

Syntax

```
value << n
```

Parameters

value	int: the value to shift
n	int: the number of places to shift left

>> (right shift)

Shifts bits to the right. The number to the left of the operator is shifted the number of places specified by the number to the right. Each shift to the right halves the number, therefore each right shift divides the original number by 2. Use the right shift for fast divisions or to extract an individual number from a packed number. Right shifting only works with integers or numbers which automatically convert to an integer such at byte and char.

```
int m = 8 >> 3;   // In binary: 1000 to 1
print(m);  // Prints "1"
int n = 256 >> 6;  // In binary: 100000000 to 100
print(n);  // Prints "4"
int o = 16 >> 3;   // In binary: 10000 to 10
print(o);  // Prints "2"
int p = 26 >> 1;   // In binary: 11010 to 1101
print(p);  // Prints "13”
```

Syntax

```
value >> n
```

Parameters

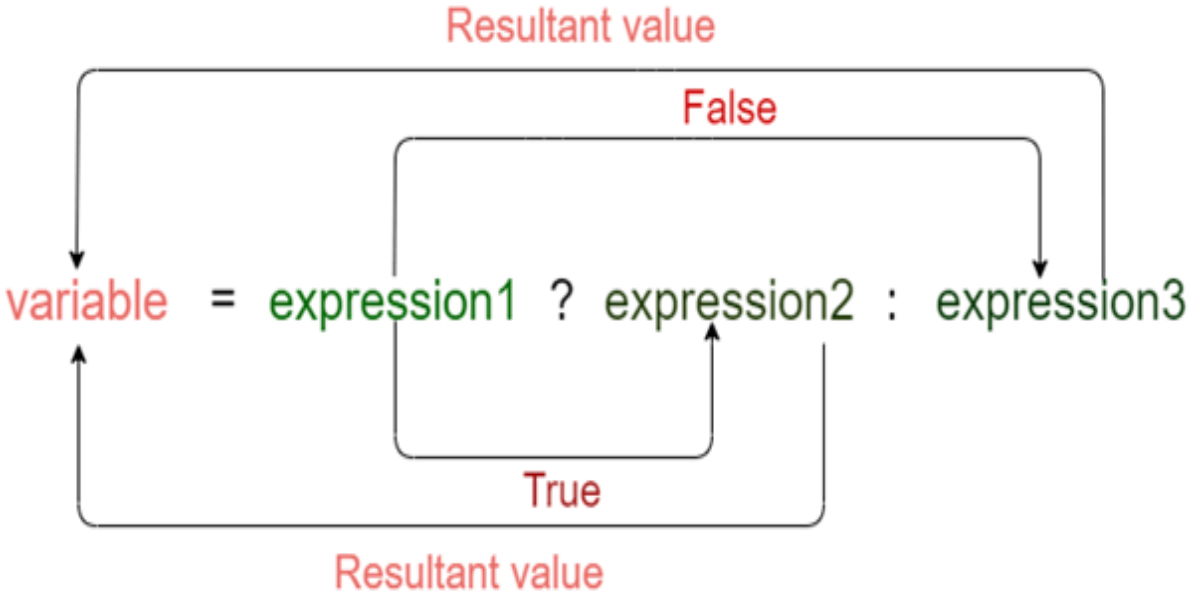
value	int: the value to shift
n	int: the number of places to shift right

6. Conditional Operators

- Conditional operators return one value if condition is true and returns another value is condition is false.
- This operator is also called as ternary operator.
  - Syntax : (Condition**?** true_value **:** false_value);
  - Example : (A > 100 ? 0 : 1);
- In above example, if A is greater than 100, 0 is returned else 1 is returned.
- This is equal to if else conditional statements.


```
#include <stdio.h>
 int main()
{
    int x=1, y;
    y = ( x ==1 ? 2 : 0 ) ;
    printf("x value is %d\n", x);
    printf("y value is %d", y);
}
```

Output:
x value is 1
y value is 2



Meaning of the above syntax.

- In the above syntax, the expression1 is a Boolean condition that can be either true or false value.
- If the expression1 results into a true value, then the expression2 will execute.
- The expression2 is said to be true only when it returns a non-zero value.
- If the expression1 returns false value then the expression3 will execute.
- The expression3 is said to be false only when it returns zero value.

```
#include <stdio.h>

int main()

{

   int age;  // variable declaration

   printf("Enter your age");

   scanf("%d",&age);   // taking user input
for age variable

   (age>=18)? (printf("eligible for voting")) :
(printf("not eligible for voting"));  //
conditional operator

   return 0;

}
```

7. Increment/Decrement Operators

- Increment/decrement Operators in C:
  - Increment operators are used to increase the value of the variable by one and
  - decrement operators are used to decrease the value of the variable by one in C programs.
- Increment operator:
  - ++var_name : pre-increment operator
  - var_name++ : post-increment operator
- Decrement operator:
  - - -var_name : pre-decrement operator
  - var_name - - : post-decrement operator
- Example:

  Increment operators : ++ i ; i ++ ;

  Decrement operators : -- i ; i -- ;


```
//Example for increment operators
 #include <stdio.h>
int main()
{
    int i=1;
    while(i<10)
    {
       printf("%d ",i);
       i++;
    }
 }
Output:1 2 3 4 5 6 7 8 9
```

```
//Example for decrement operators
 #include <stdio.h>
int main()
{
   int i=20;
   while(i>10)
   {
       printf("%d ",i);
       i--;
   }
 }
Output:20 19 18 17 16 15 14 13 12 11
```

Consider the following

m = 5;
y = ++m; (prefix)

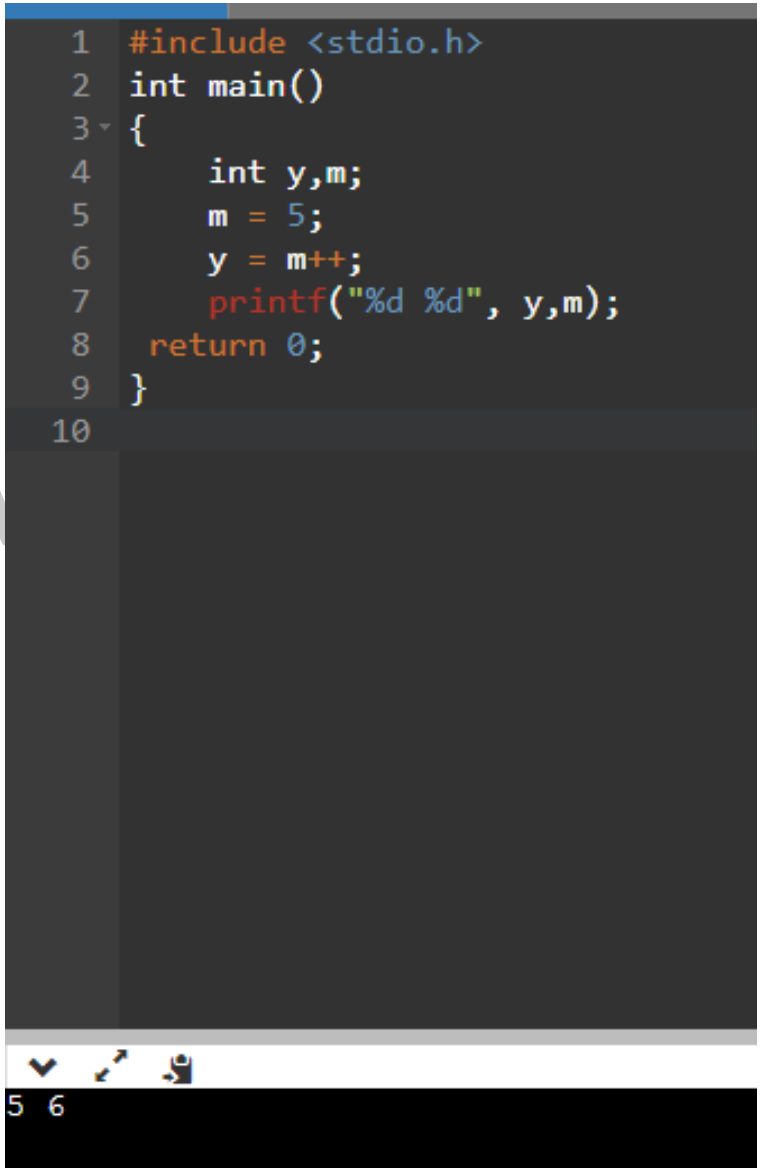
In this case the value of y and m would be 6

Suppose if we rewrite the above statement as

m = 5;
y = m++; (post fix)

Then the value of y will be 5 and that of m will be 6.

A prefix operator first adds 1 to the operand and then the result is assigned to the variable on the left. On the other hand, a postfix operator first assigns the value to the variable on the left and then increments the operand.



8. Special Operators

- Below are some of the special operators that the C programming language offers.

Operators	Description
&	This is used to get the address of the variable. Example : &a will give address of a.
*	This is used as pointer to a variable. Example : * a *where*, * is pointer to the variable a.
Sizeof ()	This gives the size of the variable. Example : sizeof (char) will give us 1.

```
#include <stdio.h>
int main()
{
    int *ptr, q;
   q = 50;
 /* address of q is assigned to ptr */
   ptr = &q;
 /* display q's value using ptr variable */
   printf("%d", *ptr);
 return 0;
}
```

Output =50

```
#include <stdio.h>
#include <limits.h>
 int main()
{
   int a;
   char b;
   float c;
   double d;
   printf("Storage size for int data type:%d \n", sizeof(a));
   printf("Storage size for char data type:%d \n",
sizeof(b));
   printf("Storage size for float data type:%d \n",
sizeof(c));
   printf("Storage size for double data type:%d\n",
sizeof(d));
  return 0;
 }
```

Outputs:

Storage size for int data type:4
Storage size for char data type:1
Storage size for float data type:4
Storage size for double data type:8

Operators Precedence in C

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left
Comma	,	Left to right

Operator Precedence and Associativity

Rules for evaluation of expression

1. First parenthesized sub expression left to right are evaluated.
2. If parenthesis are nested, the evaluation begins with the innermost sub expression.
3. The precedence rule is applied in determining the order of application of operators in evaluating sub expressions.
4. The associability rule is applied when two or more operators of the same precedence level appear in the sub expression.
5. Arithmetic expressions are evaluated from left to right using the rules of precedence.
6. When Parenthesis are used, the expressions within parenthesis assume highest priority.

Thank You